

Atividade Prática 01 – Desenvolvimento de Middleware Distribuído

Disciplina: Sistemas Distribuídos

Tema: Middleware e Comunicação Distribuída

1. Objetivo

O objetivo desta atividade é compreender o papel do middleware em Sistemas Distribuídos por meio do desenvolvimento de uma aplicação prática. Cada grupo será responsável por implementar um tipo específico de middleware, explorando conceitos como comunicação distribuída, desacoplamento, serialização de mensagens, comunicação síncrona e/ou assíncrona, escalabilidade e tolerância a falhas. A atividade servirá como base para o projeto da disciplina, que será desenvolvido posteriormente utilizando serviços da AWS.

2. Objetivos de Aprendizagem

- Compreender o papel do middleware em Sistemas Distribuídos.
- Implementar um serviço intermediário entre aplicações.
- Desenvolver APIs ou serviços distribuídos.
- Utilizar protocolos de comunicação.
- Documentar uma arquitetura distribuída.
- Avaliar vantagens e limitações da solução desenvolvida.

3. Organização da Atividade

A turma está dividida em oito grupos. Cada grupo desenvolverá um único projeto correspondente ao tema recebido.

4. Tema de Cada Grupo

Grupo	Middleware	Objetivo
G1	API Gateway	Gateway para autenticação, roteamento e balanceamento.
G2	RPC / gRPC	Chamadas remotas entre cliente e servidor.
G3	Message Queue	Comunicação assíncrona utilizando RabbitMQ ou ActiveMQ.
G4	Publish-Subscribe	Sistema baseado em eventos utilizando MQTT ou Kafka.
G5	Service Discovery	Descoberta dinâmica de serviços.
G6	Object Request Broker	Invocação remota de objetos.
G7	Middleware para IoT	Gateway IoT com ESP32, MQTT e AWS IoT Core.

G8	Middleware para Microsserviços	Comunicação entre microsserviços com API Gateway.
----	--------------------------------	---

5. Cenário de Aplicação

Cada grupo deverá escolher um cenário para contextualizar sua solução, como Hospital Inteligente, Agricultura de Precisão, Smart City, Universidade Inteligente, Biblioteca Digital, Sistema Bancário, Aeroporto, Logística, Hotel Inteligente, Indústria 4.0, Marketplace, E-commerce, Streaming de Vídeo, Rede Social, Monitoramento Ambiental ou Controle de Frotas.

6. Tecnologias

Linguagens: Java, Python, Go, C#, Node.js ou Rust.

Tecnologias: REST, gRPC, RabbitMQ, Kafka, MQTT, ZeroMQ, Redis Streams, Docker e Docker Compose.

7. Arquitetura Mínima

Cliente



Middleware



Servidor



Banco de Dados

8. Requisitos Obrigatórios

- Comunicação entre cliente e servidor
- Middleware desenvolvido pelo grupo
- Autenticação simples (JWT ou API Key)
- Registro de logs e timestamp
- Tratamento de exceções e timeout
- Documentação da API (Swagger/OpenAPI quando aplicável)
- Testes funcionais
- Repositório GitHub
- README com instruções de execução

9. Entregáveis

- Código-fonte completo
- Repositório GitHub
- Slides da apresentação
- Documento descrevendo a arquitetura
- Apresentação por grupo das atividades 1 e 2 na semana do dia 27/07 com 15 minutos para cada grupo.

10. Integração com AWS

- G1 → Amazon API Gateway
- G2 → Comunicação via gRPC
- G3 → Amazon SQS
- G4 → Amazon SNS / AWS IoT Core
- G5 → AWS Cloud Map
- G6 → Comunicação entre serviços distribuídos
- G7 → AWS IoT Core + Lambda
- G8 → ECS/EKS + API Gateway + CloudWatch + IAM

Não é necessário implementar a solução na AWS nesta atividade; basta apresentar a arquitetura proposta.

11. Critérios de Avaliação

Critério	Peso
Fundamentação teórica	15%
Arquitetura da solução	20%
Implementação do middleware	30%
Qualidade do código e documentação	15%
Demonstração prática	10%
Integração com AWS e respostas às perguntas	10%